StampFly 勉強会 2026

ドローン制御工学ハンズオン ワークショップ

伊藤 恒平

2026/3/2 – 3/6

Lesson 0

環境セットアップ

Environment Setup

StampFly Workshop

ワークショップカリキュラム / Workshop Curriculum

Day	Lesson	テーマ
Day 1	0–2	環境セットアップ + モータ + コントローラ
Day 2	3–5	LED + IMU + 初フライト
Day 3	6–8	PID 制御 + テレメトリ + モデリング
Day 4	9–11	姿勢推定 + Python SDK + 競技会準備
Day 5 AM		ホバリングタイム競技会

StampFly ハードウェア / Hardware

項目	仕様
MCU	ESP32-S3 (M5Stamp S3)
IMU	BMI270 (6 軸 400 Hz)
気圧	BMP280
ToF	VL53L3CX
質量	37 g（バッテリ含む）
通信	ESP-NOW + WiFi
バッテリ	LiPo 1S 3.7 V

エコシステム全体像 / Ecosystem Overview

`tools/` 開発ツール	sf CLI
`analysis/` データ解析	Jupyter / Python
`simulator/` シミュレータ	Genesis 3D 物理
`control/` 制御設計	PID, MPC, SIL
`firmware/` 組込みコード	ESP32-S3 / ESP-IDF / C++
`protocol/` 通信仕様	SSOT: YAML → コード自動生成

- `firmware/` — 組込みコード
- `control/` — 制御設計
- `simulator/` — 3D シミュレータ
- `analysis/` — データ解析
- `tools/` — sf CLI
- `protocol/` — 通信仕様

開発ツール sf CLI / Development Tool

カテゴリ	コマンド例	説明
ビルド	`sf build` / `sf flash`	ファームウェア開発
診断	`sf doctor` / `sf monitor`	デバッグ
ログ	`sf log wifi` / `sf log viz`	テレメトリ取得・可視化
シミュレータ	`sf sim run`	仮想環境で練習
キャリブレーション	`sf cal gyro/accel/mag`	センサ校正

ワークフロー

`sf lesson switch N` → `sf lesson build` → `sf lesson flash`

Windows 環境構築 (1/2) / Windows Setup

確認コマンド (CMD)	期待される結果	なければ
`git --version`	git version 2.x	`winget install Git.Git`
`python --version`	Python 3.10 以上	`winget install Python.Python.3.12`

インストール手順（CMD で実行）

1. リポジトリをクローン
2. `install.bat` を実行（ESP-IDF + sfcli 自動インストール）
3. `setup_env.bat` で開発環境をアクティベート

```
git clone https://github.com/.../stampfly-ecosystem.git
cd stampfly-ecosystem
install.bat
setup_env.bat
```

Windows 環境構築 (2/2) / Driver & Verification

USB シリアルドライバ

CH9102F ドライバをインストール（M5Stack 製品用）

`https://docs.m5stack.com/en/download`

動作確認

`sf doctor` で環境診断 — すべて OK になれば完了

macOS / Linux ユーザー

`install.sh` + `setup_env.sh` を使用してください

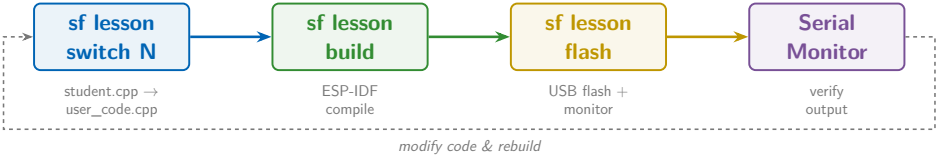
今日のゴール / Today's Goal

目標

ワークショップファームウェアをビルド・書き込み・動作確認する

1. `sf lesson build` でビルド
2. `sf lesson flash` で書き込み
3. シリアルモニタで `"Hello StampFly!"` を確認

開発フロー / Development Flow



sf CLI とは / What is sf CLI?

- StampFly 開発専用のコマンドラインツール
- ESP-IDF のコマンドをシンプルに実行できる

コマンド	説明
`sf doctor`	環境診断
`sf lesson build`	ワークショップビルド
`sf lesson flash`	書き込み＋モニタ
`sf lesson switch N`	レッスン切り替え
`sf monitor`	シリアルモニタ

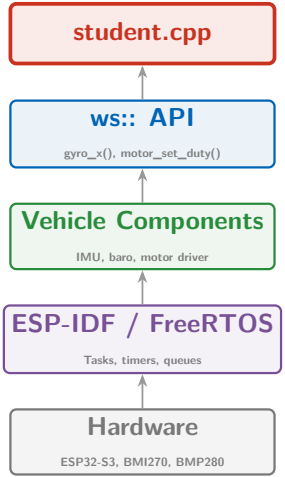
ワークショップの構造 / Workshop Structure

学生が実装する関数は 2 つだけ！

- `setup()` — 起動時に 1 回呼ばれる
- `loop_400Hz(dt)` — 400Hz（2.5ms 毎）で呼ばれる

- ハードウェアの複雑さは `ws::` 名前空間で隠蔽
- FreeRTOS, SPI/I2C, センサーフュージョンを意識する必要なし

ファームウェアの全体像 / Firmware Architecture



← You are here!

setup() / loop_400Hz()

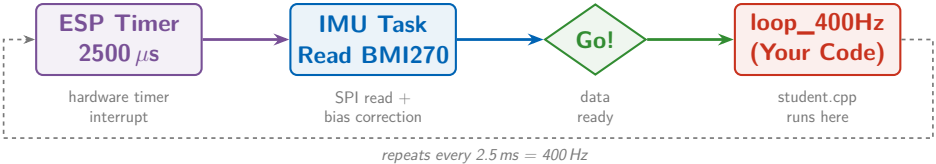
sensor read, filter, motor

xTaskCreate, xQueueSend

SPI, I2C, GPIO

- 皆さんのコードはここ！
  `student.cpp` を編集するだけ
- `ws::gyro_x()` と書くだけで
  SPI 通信→フィルタ→バイアス補正を
  全部やってくれる
- 下のレイヤーはワークショップ中
  意識する必要なし

400Hz ループの裏側 / Behind the 400Hz Loop



ポイント

`loop_400Hz(dt)` は常に IMU データ更新後に呼ばれる— 学生はタイマー管理不要、ただ関数を書くだけ！

実習: Hello StampFly / Hands-on

```cpp
#include "workshop_api.hpp"

void setup()
{
    ws::print("Hello StampFly!");
}

void loop_400Hz(float dt)
{
    // Nothing to do in Lesson 0
}
```

student.cpp

手順

1. `sf lesson switch 0`
2. 上のコードを `student.cpp` に書く
3. `sf lesson build && sf lesson flash`

作業の進め方 / Workflow

ファイル構造

```
firmware/workshop/
  ├── lessons/
  │   ├── lesson_00/student.cpp
  │   ├── lesson_01/student.cpp
  │   └── ...
  └── main/
      └── user_code.cpp ← 編集！
```

1. `sf lesson switch N`
   テンプレートを `user_code.cpp` にコピー
2. `user_code.cpp` を編集
3. `sf lesson build`
4. `sf lesson flash`

注意

`student.cpp` はテンプレートなので直接編集しない！
`switch` で何度でもリセットできる

コントローラのセットアップ / Controller Setup

コントローラのビルドと書き込み

1. コントローラを USB 接続
2. `sf build controller`
3. `sf flash controller`

確認

LCD に起動画面が表示されれば OK
Lesson 2 でペアリングして使用する

シミュレータで遊ぶ / Try the Simulator

コマンド

```
sf sim run
```

デフォルト（VPython + ボクセルワールド）

```
sf sim run -w ringworld
```

軽量ワールド

操作	説明
スロットル	上昇
ロール / ピッチ	傾き
ヨー	回転

ヒント

コントローラを USB 接続すればジョイスティックで操縦可能！
アクロモード（角速度制御）で飛行 → まずは感覚をつかもう

チェックポイント / Checkpoint

確認事項

- ☐ `sf doctor` がエラーなしで通る
- ☐ ビルドが成功し `"Hello StampFly!"` が表示される
- ☐ コントローラのビルド・書き込みが完了した
- ☐ シミュレータが起動し操縦を試した

次のレッスン

Lesson 1: モータ制御 — モータを回してプロペラの動きを確認

Lesson 1

モータ制御

Motor Control

StampFly Workshop

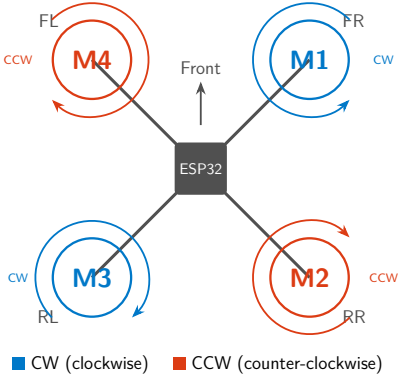
今日のゴール / Today's Goal

目標

モータの番号と配置を理解し、PWM で個別制御する

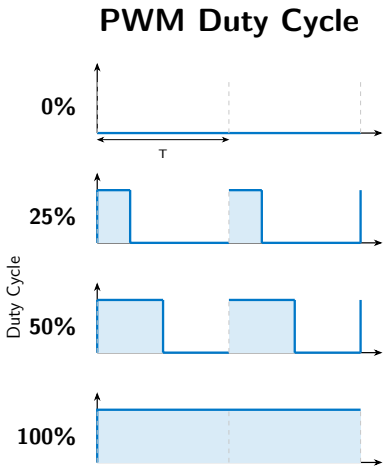
- モータ番号 M1–M4 の位置を覚える
- PWM デューティ比でモータ回転数を制御
- 安全な地上テストの方法を学ぶ

モータ配置 / Motor Layout



- 対角のモータが同じ方向に回転（トルクバランス）
- M1(FR), M3(RL) = CW / M2(RR), M4(FL) = CCW

PWM とは / What is PWM?



- **Pulse Width Modulation**（パルス幅変調）
- **Duty 0%** = 停止
- **Duty 100%** = 最大回転
- API: `motor_set_duty(id, duty)`
  id=1–4, duty=0.0–1.0

モータ制御 API

関数	説明	引数
`motor_set_duty(id, duty)`	個別モータ設定	id=1–4, duty=0.0–1.0
`motor_set_all(duty)`	全モーター括	duty=0.0–1.0
`motor_stop_all()`	全モータ停止	—

安全注意 / Safety

duty は 0.15（15%）以下で！ 機体を手で押さえてテスト！

実習: モータを順番に回す / Hands-on

```cpp
#include "workshop_api.hpp"
static uint32_t timer = 0;

void setup() {
    ws::print("Lesson 1: Motor Control");
}
void loop_400Hz(float dt) {
    timer++;
    // 800 ticks = 2 seconds at 400Hz
    int motor_id = (timer / 800) % 4 + 1;
    ws::motor_stop_all();
    ws::motor_set_duty(motor_id, 0.1f);
}
```

student.cpp

チェックポイント / Checkpoint

確認事項

- ☐ M1 が FR（右前）のプロペラを回す
- ☐ M1→M2→M3→M4 の順に回転する
- ☐ 各モータ 2 秒ごとに切り替わる

次のレッスン

Lesson 2: コントローラ入力 — スティックでモータを操作

Lesson 2
コントローラ入力
Controller Input

StampFly Workshop

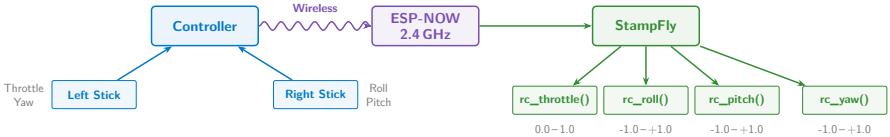
今日のゴール / Today's Goal

目標: スティック値を変数に読み取り、演算でモータを個別制御する

- ESP-NOW 無線通信の仕組みを理解
- 変数と四則演算でスティック → モータ Duty を計算
- オープンループ手動操縦の限界を体感

ESP-NOW 通信フロー / Communication Flow

Controller → ESP-NOW → StampFly API



コントローラ API

関数	説明	値域
`rc_throttle()`	スロットル	0.0 – 1.0
`rc_roll()`	ロール	-1.0 – +1.0
`rc_pitch()`	ピッチ	-1.0 – +1.0
`rc_yaw()`	ヨー	-1.0 – +1.0
`is_armed()`	ARM 状態	true / false
`motor_set_duty(id, v)`	モータ個別制御	id=1–4, v=0.0–1.0

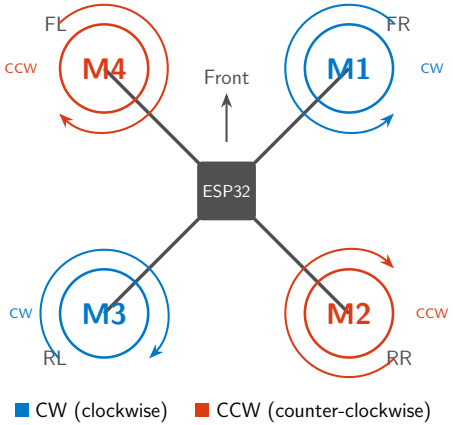
ペアリング手順 / Pairing

Step	操作
1	コントローラの **M5 ボタンを押しながら電源 ON** → LCD に “Pairing mode...” 表示
2	StampFly のボタンを **3 秒長押し** → 青 LED 高速点滅 + ビープ音
3	ペアリング完了 → ビープ音が鳴り、次回から自動接続

注意

- 教室では**一組ずつ**ペアリングする（ブロードキャスト通信のため近くの機体と干渉する可能性）
- うまくいかない場合: 両方を再起動して Step 1 からやり直す

モータ配置とミキシング / Motor Layout & Mixing



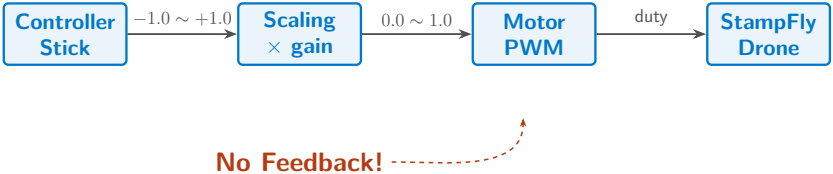
Motor	T	Roll	Pitch	Yaw
M1 FR	+	+	−	−
M2 RR	+	+	+	+
M3 RL	+	−	+	−
M4 FL	+	−	−	+

直感的に理解する

Pitch+ → 前傾 → 前 (M1,M4) 弱 + 後 (M2,M3) 強
Roll+ → 右傾 → 左 (M3,M4) 弱 + 右 (M1,M2) 強

オープンループ制御 / Open-Loop Control

Open-Loop Control



問題点

フィードバックがないため、外乱（風・重心ずれ）に対応できない！
→ Lesson 5–6 で PID 制御を導入して解決

実習: 手動ミキシング / Hands-on: Manual Mixing

```cpp
#include "workshop_api.hpp"
static uint32_t tick = 0;
void setup() { ws::print("L2: Open-Loop Control"); }
void loop_400Hz(float dt) {
    tick++;
    float t = ws::rc_throttle();
    float r = ws::rc_roll()  * 0.3f;
    float p = ws::rc_pitch() * 0.3f;
    float y = ws::rc_yaw()   * 0.3f;
    ws::motor_set_duty(1, t + r - p - y); // FR
    ws::motor_set_duty(2, t + r + p + y); // RR
    ws::motor_set_duty(3, t - r + p - y); // RL
    ws::motor_set_duty(4, t - r - p + y); // FL
    if (tick % 80 == 0)
        ws::print("T=%.2f R=%.2f P=%.2f Y=%.2f",
            t, r, p, y);
}
```

student.cpp

チェックポイント / Checkpoint

確認事項

- ☐ コントローラとペアリングできた
- ☐ スロットルで全モータが均等に回る
- ☐ ピッチスティックで前後モータの回転差が出る
- ☐ ゲイン (0.3f) を変えて挙動の違いを確認した

次のレッスン

Lesson 3: LED 制御 — システム状態を LED で可視化